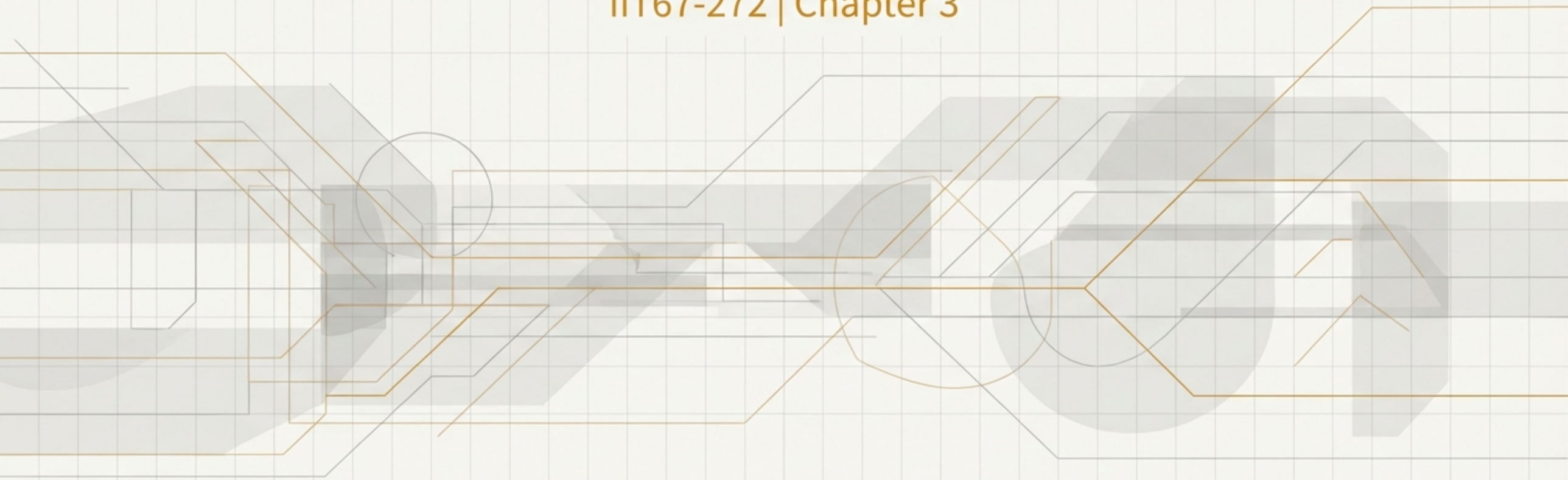
The Language of the Web: A Guide to RESTful API Design

IIT67-272 | Chapter 3



Walailak University – IIT67-272

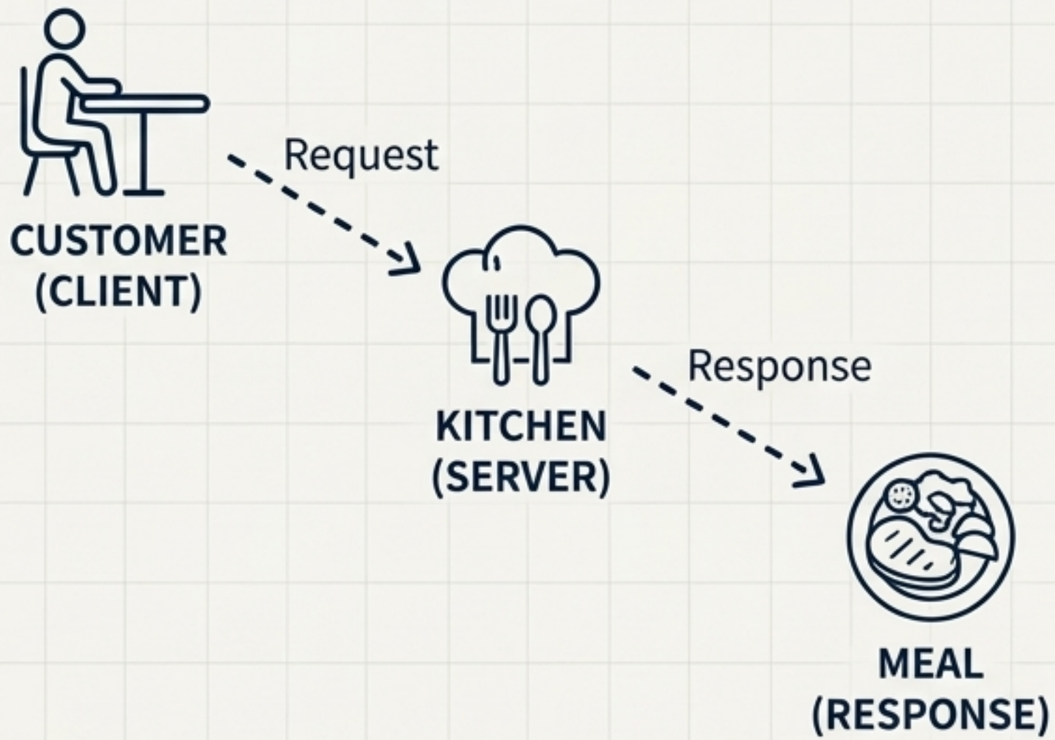
Your Goal for the Next 90 Minutes

Grasp the core concepts of HTTP and the architectural principles of REST. This knowledge is the foundation for the hands-on API programming you will do in the next workshop.

PART 1: THE FOUNDATION

HTTP: The Protocol That Powers the Web

- **HTTP**: Stands for **Hypertext Transfer Protocol**. It's the fundamental protocol for communication on the web.
- **Client-Server Model**: Communication happens between two parties:
  - **Client**: The one making the request (e.g., your web browser, a mobile app, Postman).
  - **Server**: The one holding the resources, which processes the request and sends back a response.
- **Stateless**: Every request is treated as a brand new, independent interaction. The server doesn't remember past requests.



HTTP enables interaction between frontend and backend systems.

Every Interaction is a Request-Response Cycle

CLIENT REQUEST

- Method (e.g., GET)
- URL (e.g., /students)
- Headers (e.g., Content-Type)
- Body (Optional, for POST/PUT)

SERVER RESPONSE

- Status Code (e.g., 200 OK)
- Headers (e.g., Content-Type)
- Body (The requested data, e.g., JSON)

Communication on the web *always* follows this Request → Response pattern.

PART 2: THE VOCABULARY

The Verbs of the Web: Common HTTP Methods

GET

Retrieve data.

`GET /students`

POST

Create new data.

`POST /students`

PUT

Update an *entire* record.

`PUT /students/1`

PATCH

Update a *partial* record.

`PATCH /students/1`

DELETE

Remove data.

`DELETE /students/1`

Think **CRUD**: **C**reate, **R**ead, **U**pdate, **D**elete maps directly to **POST**, **GET**, PUT/PATCH, **DELETE**.

Understanding the Server's Reply: HTTP Status Codes

SUCCESS (2xx)

200 OK: The request succeeded. (e.g., Data successfully retrieved).

201 Created: A new resource was successfully created. (Typically after a POST).

CLIENT ERROR (4xx)

400 Bad Request: The server couldn't understand the request due to invalid syntax. (e.g., Validation error).

401 Unauthorized: Authentication is required and has failed or has not been provided.

404 Not Found: The server can't find the requested resource.

SERVER ERROR (5xx)

500 Internal Server Error: The server encountered an unexpected condition that prevented it from fulfilling the request.

Always return clear and consistent status codes. They are the primary way an API communicates success or failure to the client.

PART 3: THE GRAMMAR

REST: An Architectural Style for Building Web APIs

REST stands for **Representational State Transfer**.
It's not a protocol, but a set of architectural principles for designing networked applications.

The Key Principles of REST

1. Client-Server Separation

Clients and servers evolve independently.

2. Stateless

Each request from a client contains all the information needed to understand and process the request.

3. Uniform Interface

A consistent way of interacting with the server (using HTTP methods, resource-based URLs, and JSON/XML representations).

4. Resource-Based

APIs are designed around resources (the "nouns" of the system).

The Core Principle of REST: Focus on Resources, Not Actions

Use **nouns** for endpoint names, not verbs. The HTTP method already specifies the action.

RESTful Way (Correct)

```
GET /students   // Get all students
POST /students  // Create a student
GET /students/5 // Get student with ID 5
PUT /students/5 // Update student with ID 5
```

Action-Oriented Way (Incorrect)

```
GET /getAllStudents
POST /createNewStudent
GET /getStudentById?id=5
PUT /updateStudent
```

Designing with **nouns** makes your API more predictable, stable, and easier for other developers to understand. The resource (`/students`) stays the same, while the action (`GET`, `POST`) changes.

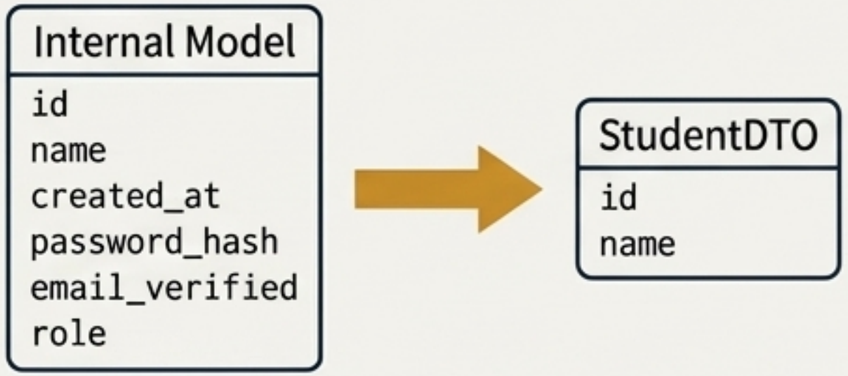
Blueprint: A RESTful API for Student Management

Operation	HTTP Method	Endpoint	Description
Create a new student	**POST**	`/students`	Adds a new student record to the collection.
Retrieve all students	**GET**	`/students`	Returns a list of all student records.
Retrieve a single student	**GET**	`/students/{id}`	Finds and returns a single student by their ID.
Update a student	**PUT**	`/students/{id}`	Modifies an existing student's data.
Delete a student	**DELETE**	`/students/{id}`	Removes a student record from the collection.

Ensuring Data Integrity: DTOs & Validation

Data Transfer Objects (DTOs)

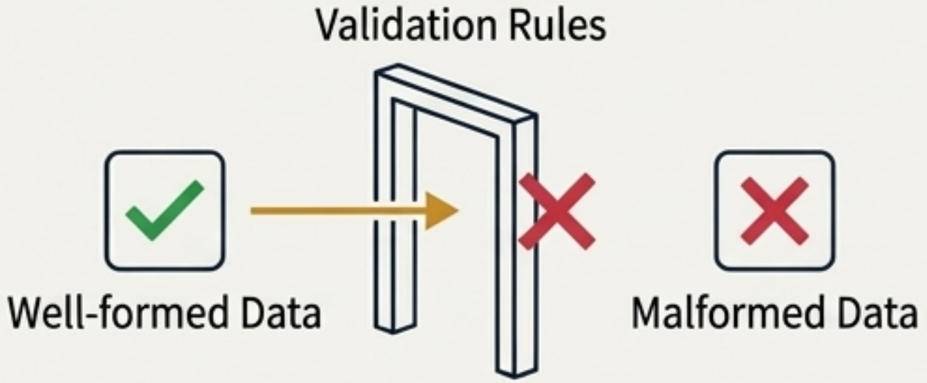
What it is: A DTO is an object that defines how data will be sent over the network.
Why use it: It decouples your internal database model from the API, ensuring you only expose the necessary data and protecting your internal structure.



DTOs protect your internal model and simplify data contracts.

Input Validation

What it is: The process of ensuring any data sent to the API is correct and well-formed before it's processed.
Why use it: It prevents invalid data from entering your system, improving reliability and security.



Example: Using backend framework annotations like `@NotNull`, `@Email`, `@Size` to automatically enforce rules.

PART 4: THE PRACTICE

Meet Your API Toolkit: Postman

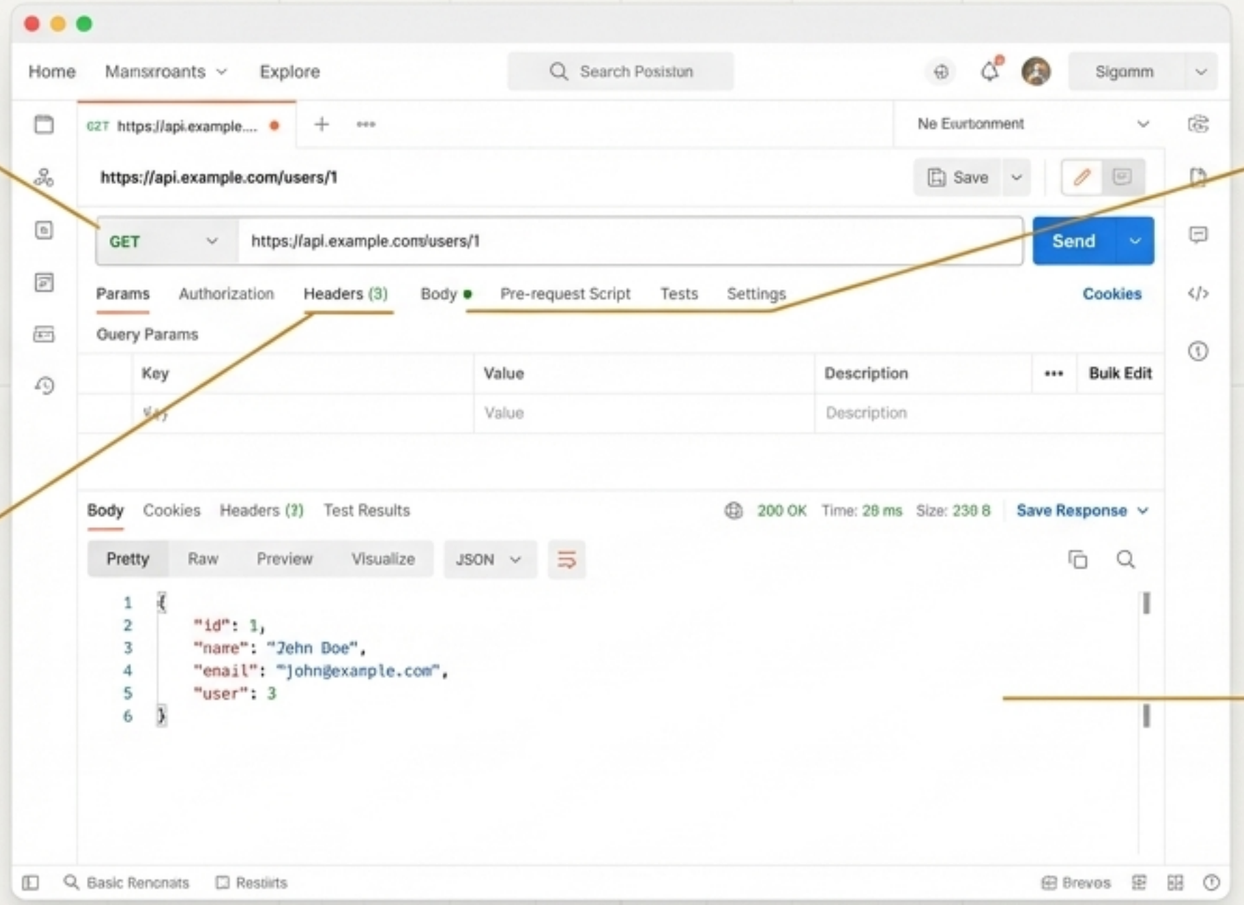
Postman is an industry-standard application for building, testing, and debugging APIs. It allows you to manually craft and send any kind of HTTP request and inspect the response in detail.

Key Uses:

- Testing endpoints with different methods (GET, POST, etc.).
- Sending data (JSON bodies) with requests.
- Managing authentication (headers and tokens).
- Inspecting server responses, status codes, and data.

A Quick Tour of the Postman Interface

Request Builder: Choose your HTTP Method (GET, POST, etc.) and enter the endpoint URL here.



Headers Tab: Define metadata for your request, like authentication tokens or content type.

Body Tab: For POST or PUT requests, you provide the data (e.g., in JSON format) here.

Response Panel: After sending the request, the server's response, status code, and data appear here.

A Test Flight: A Real CRUD Workflow in Postman

Step 1: Create a Resource

Method: `POST`

URL: `/students`

Body:
```
{
  "name": "John Doe",
  "major": "Computer Science"
}
```

Expected Response: `201 Created`

Step 2: Verify the Resource

Method: `GET`

URL: `/students`

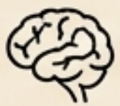
Expected Response: `200 OK` with a JSON array containing John Doe.

Step 3: Delete the Resource

Method: `DELETE`

URL: `/students/{id}` (using the ID from the previous step)

Expected Response: `200 OK` or `204 No Content`



Your turn in the workshop: Test all the `/students` endpoints using these different HTTP methods.

You Are Now Ready for the Workshop

You have learned the language and grammar of modern web APIs.

- ☑ The fundamentals of the **HTTP** protocol.
- ☑ The core **Methods** and **Status Codes** for API communication.
- ☑ The principles of **RESTful API design**.
- ☑ How to test and debug APIs with **Postman**.

🚀 You are now ready to build a CRUD API in the next workshop.